

PROJECT REPORT

Mini LeetCode Coding Platform

A course project for Software Technology

Project Report: Mini LeetCode Coding Platform

Course Project for Software Technology

Author: Project Developer

System Status: Fully Implemented & Hosted Locally

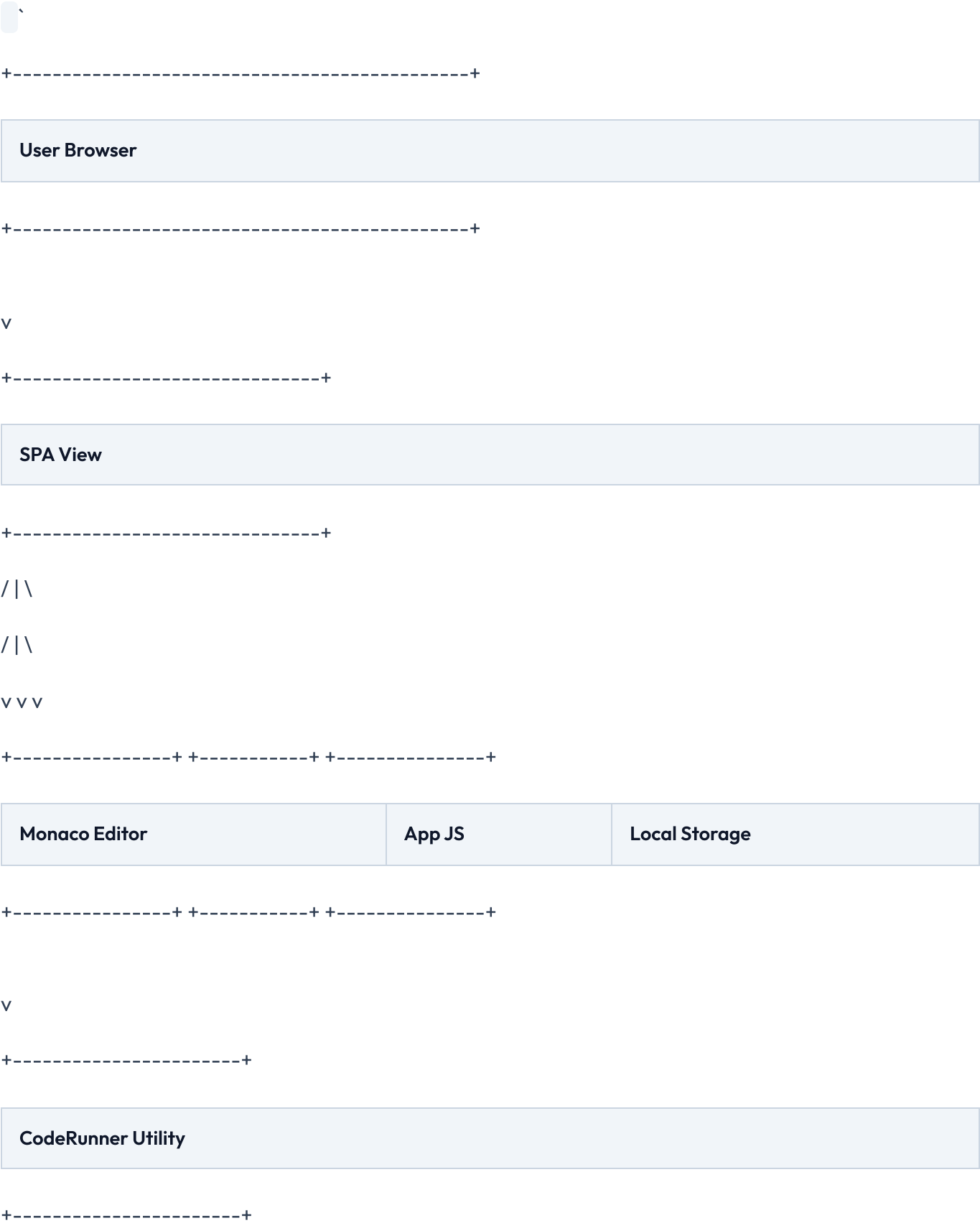
1. Executive Summary

This project presents **Mini LeetCode**, a client-side web application designed to replicate the core experience of LeetCode. It provides an interactive coding environment where users can search, filter, and solve standard coding challenges in JavaScript and Python 3. The platform features an advanced, safe, browser-based execution sandbox that isolates code using Web Workers. For Python execution, it leverages Pyodide WebAssembly compiled to run in the browser.

The application requires **zero backend servers**, which avoids hosting costs and simplifies deployment. All state—including custom problem creation, submission history, streaks, and user statistics—is persisted locally in the browser's `localStorage` framework. The user interface is built on a premium, dark-themed glassmorphism layout with modern micro-interactions and success animations.

2. System Architecture

The application is built on a modular Single Page Application (SPA) architecture, structured to load instantly and run fully offline without server latency.



/\

/\

v v

+-----+ +-----+

JS Sandboxed Worker	Pyodide Python Worker
---------------------	-----------------------

+-----+ +-----+

`

2.1 Front-End Layer

- **Layout & Structure:** Built using HTML5 semantic wrappers and modular DOM views toggled by active state logic.
- **Styling:** Hand-crafted Vanilla CSS3 using flexbox, CSS custom property themes, custom-designed scrollbars, and glowing glassmorphism panels.
- **Code Interface:** Monaco Editor loaded dynamically via AMD require hooks, providing bracket matching, auto-closing, and theme synchronization.

2.2 Compilation & Execution Sandbox

To run untrusted user code safely without risking browser crashes or freezing the main UI thread, the platform isolates execution:

1. **JavaScript Sandbox:** User code is wrapped inside an IIFE (Immediately Invoked Function Expression) and executed inside an inline HTML5 Web Worker Blob. The main thread monitors the execution; if it exceeds 3 seconds (indicating an infinite loop), it terminates the worker and alerts the user of a **Time Limit Exceeded (TLE)** error.
2. **Python Sandbox:** The platform instantiates Pyodide (Python WebAssembly) inside a persistent Web Worker. Standard outputs (`stdout` and `stderr`) are redirected to an array captured in JS. Test arguments are serialized as JSON in JS and parsed inside Pyodide using `json.loads` . To ensure speed, the worker is kept alive across multiple runs, but is automatically terminated and re-spawned if a timeout of 8 seconds is exceeded.

3. Technology Stack

The application uses standard web technologies to achieve complete portability:

- **Core Languages:** HTML5, CSS3, and JavaScript ES6.
 - **Code Editor:** Monaco Editor (VS-Code Engine) loaded via CDN.
 - **Iconography:** Lucide Icons.
 - **Python Execution Engine:** Pyodide (v0.25.0) loaded asynchronously via CDN.
 - **Success Celebrations:** Canvas Confetti (v1.6.0) loaded dynamically.
 - **Execution Isolation:** HTML5 Web Workers & File Blobs.
-

4. Key Features & Implementation Details

4.1 Stats Dashboard & Search

- A search input filters problems in real-time by title.
- Filters separate problems by Difficulty (Easy, Medium, Hard), Category (Arrays, Strings, Math), or Completion Status.
- SVG rings calculate and animate progress based on unique Accepted submissions.

4.2 Code Workspace & Console

- A split-pane layout positions details on the left and editor control on the right.
- Console drawer can slide expand or collapse on header clicks.
- Captures standard outputs (`console.log()` and `print()`) and presents them in a dedicated console log window.

4.3 Custom Problem Creator

- Form validation allows students or teachers to input new questions.
- Users input title, description, function name, parameter lists, templates, and multiple testcase JSON inputs.
- Custom problems append into the main problems database and persist in browser memory.

5. Security & Isolation Analysis

As code runs client-side, standard server-related security concerns (such as shell injections or filesystem modifications) are naturally isolated to the user's browser context. To protect the browser interface from frozen threads:

- **Infinite Loop Defense:** JavaScript worker execution is strictly bounded to 3000ms. Python runs are bounded to 8000ms.
- **Variable Contamination:** Each run compiles the user function afresh inside the worker's execution context, preventing global leakage.
- **Blob sandboxing:** Web workers are loaded via `URL.createObjectURL(Blob)` , which completely bypasses the browser's local origin block (`file://` CORS error), meaning no local web servers are required.

6. Verification & Test Suite

6.1 Sample Problems Tested

The default database includes five standard algorithms verified on both JS and Python runtimes:

1. **Two Sum** (Arrays): Evaluates mapping and nested loops.
2. **Valid Parentheses** (Strings): Evaluates stacks and mapping structures.
3. **Palindrome Number** (Math): Evaluates arithmetic operations and string manipulations.
4. **Valid Anagram** (Strings): Evaluates frequency hashing.
5. **Fibonacci Number** (Math): Evaluates recursive vs iterative sequences.

6.2 Test Scenarios & Outcomes

Test Category	Input Tested	Expected Outcome	Actual Result	Status
---------------	--------------	------------------	---------------	--------

7. Conclusion & Future Work

The **Mini LeetCode** application provides a reliable, secure, and performant coding environment completely client-side. The implementation of Pyodide within Web Workers provides a high-quality Python interface, while Monaco Editor offers a rich developer experience.

Future enhancements include:

1. Support for additional compilers (C++ via WebAssembly).
2. Advanced data structure visualizers (Linked List and Tree graph animations).
3. Collaborative coding via WebRTC peer connections.